

AlojaME

Sistema de Gestión y Reserva de Alojamientos

Especificación de Requisitos de Software (ERS)

Materia: Metodología de Sistemas I / Programación III
Docente: Lucrecia Bazán / Rodrigo Soto

Universidad Tecnológica Nacional — Facultad Regional Mar del Plata

2026

Índice

1. Introducción
 - 1.1 Propósito
 - 1.2 Ámbito del sistema
 2. Descripción general del sistema
 - 2.1 Perspectiva del producto
 - 2.2 Objetivos del sistema
 - 2.3 Herramientas utilizadas
 3. Definición de requisitos del sistema
 - 3.1 Requisitos funcionales
 - 3.2 Requisitos no funcionales
 4. Diagrama de casos de uso
 - 4.1 Especificación de caso de uso
 5. Modelo conceptual
-

1. Introducción

Este documento presenta la Especificación de Requisitos de Software (ERS) para el sistema 'AlojaME', una plataforma web de gestión y reserva de alojamientos. Esta especificación se desarrolla basándose en el estándar IEEE (Práctica Recomendada para Especificaciones de Requisitos de Software ANSI/IEEE 830, 1998).

En el presente documento se abordará toda la documentación necesaria para la exposición detallada de los requisitos del software desarrollado, para su completo entendimiento y comprensión. El mismo consta de varias secciones que se explicarán al detalle para un conocimiento claro del propósito y uso de la aplicación web.

1.1 Propósito

El presente documento tiene como propósito definir las funcionalidades, restricciones y características del sistema 'AlojaME'. Está dirigido a:

- Desarrolladores del sistema, para guiar la implementación.
- Docentes evaluadores, para la revisión y corrección del trabajo práctico.
- Usuarios finales, para comprender las capacidades del sistema.

1.2 Ámbito del sistema

A fines prácticos, se hará referencia al sistema como 'AlojaME', 'el proyecto' o 'el sistema'.

El sistema permitirá:

- Registro e inicio de sesión de usuarios mediante correo electrónico y contraseña.
- Gestión de inmuebles (casas, departamentos, hoteles) por parte de anfitriones.
- Reserva de alojamientos por parte de clientes para un período determinado.
- Gestión de reservas: creación, consulta y cancelación.
- Registro, modificación y eliminación de reseñas sobre alojamientos.

- Administración de usuarios por parte de un rol administrador.
- Carga y gestión de imágenes de alojamientos mediante la API externa Cloudinary.

El sistema no incluirá (en versión actual):

- Sistema de pagos en línea.
- Mensajería directa entre anfitrión y cliente.
- Sistema de notificaciones en tiempo real.

Los beneficios del sistema son facilitar la administración y comunicación entre propietarios de inmuebles y potenciales clientes, centralizando la gestión de alojamientos en una interfaz web accesible desde navegadores modernos.

2. Descripción general del sistema

2.1 Perspectiva del producto

AlojaME es un sistema web independiente que implementa una arquitectura REST con backend en Spring Boot y frontend en HTML/CSS/JavaScript servido como contenido estático.

El sistema consume la API externa de Cloudinary para la gestión y almacenamiento de imágenes de los alojamientos. Cloudinary provee servicios de carga, transformación y entrega de imágenes en la nube, permitiendo a los anfitriones gestionar galerías de fotos para sus propiedades.

Dependencias principales del framework:

- Spring Security con JWT (jsonwebtoken) para manejo de credenciales y sesiones.
- Spring Web para la exposición de la API REST.
- Spring Data JPA (Hibernate) para la persistencia de datos.
- MySQL Driver para la conexión a la base de datos relacional.
- Lombok para reducción de código boilerplate.
- Springdoc OpenAPI (Swagger UI) para la documentación de la API.

2.2 Objetivos del sistema

Se espera que al finalizar el desarrollo, el sistema cumpla los siguientes objetivos:

- Permitir el registro y autenticación segura de usuarios con distinción de roles (CLIENTE, ANFITRION, ADMINISTRADOR).
- Proveer a los anfitriones herramientas para crear, editar, eliminar y gestionar sus alojamientos con soporte de galería de imágenes.
- Permitir a los clientes buscar, consultar y reservar alojamientos disponibles para fechas determinadas.
- Habilitar la gestión de reservas tanto para clientes (consulta y cancelación) como para anfitriones (consulta de reservas de sus alojamientos).
- Ofrecer un sistema de reseñas donde los clientes puedan valorar y comentar los alojamientos.
- Proveer al administrador herramientas para gestionar los usuarios registrados en el sistema.

2.3 Herramientas utilizadas

Para el desarrollo del proyecto se utilizaron las siguientes tecnologías:

- Java 21: como lenguaje principal para el backend.
 - Spring Boot: como framework para el desarrollo de la API REST.
 - Spring Security + JWT (jjwt): para la autenticación y autorización de usuarios mediante tokens.
 - Spring Data JPA (Hibernate): para la persistencia y gestión de datos.
 - MySQL: como sistema gestor de base de datos relacional.
 - HTML5, CSS3 y JavaScript (Vanilla): para el desarrollo de la interfaz web (frontend estático).
 - Cloudinary API: para el almacenamiento y gestión de imágenes en la nube (API externa consumida).
 - Swagger / Springdoc OpenAPI: para la documentación interactiva de los servicios REST.
 - Maven: para la gestión de dependencias y construcción del proyecto.
 - Git y GitHub: para el control de versiones del proyecto.
-

3. Definición de requisitos del sistema

3.1 Requisitos funcionales

Los requisitos funcionales del sistema son los siguientes:

Gestión de usuarios

- **RF01:** El sistema debe permitir al visitante registrarse como usuario.
- **RF02:** El sistema debe permitir al usuario iniciar sesión mediante correo electrónico y contraseña.
- **RF03:** El sistema debe permitir al usuario consultar su información personal.
- **RF04:** El sistema debe permitir al usuario modificar su información personal.
- **RF05:** El sistema debe permitir al administrador consultar los usuarios registrados.
- **RF06:** El sistema debe permitir al administrador eliminar usuarios registrados.

Gestión de alojamientos

- **RF07:** El sistema debe permitir al anfitrión registrar un nuevo alojamiento (casa, departamento u hotel).
- **RF08:** El sistema debe permitir al anfitrión modificar la información de sus alojamientos.
- **RF09:** El sistema debe permitir al anfitrión eliminar sus alojamientos.
- **RF10:** El sistema debe permitir al usuario consultar los alojamientos disponibles.
- **RF11:** El sistema debe permitir al usuario visualizar el detalle de un alojamiento.
- **RF12:** El sistema debe permitir al anfitrión gestionar la galería de imágenes de sus alojamientos mediante la API de Cloudinary.

Gestión de reservas

- **RF13:** El sistema debe permitir al usuario realizar una reserva de un alojamiento.
- **RF14:** El sistema debe permitir al usuario consultar sus reservas.
- **RF15:** El sistema debe permitir al usuario cancelar una reserva.
- **RF16:** El sistema debe permitir al anfitrión consultar las reservas asociadas a sus alojamientos.

Gestión de reseñas

- **RF17:** El sistema debe permitir al usuario registrar una reseña sobre un alojamiento.
- **RF18:** El sistema debe permitir al usuario modificar una reseña realizada.
- **RF19:** El sistema debe permitir al usuario eliminar una reseña realizada.
- **RF20:** El sistema debe permitir al usuario consultar las reseñas de un alojamiento.

Administración general

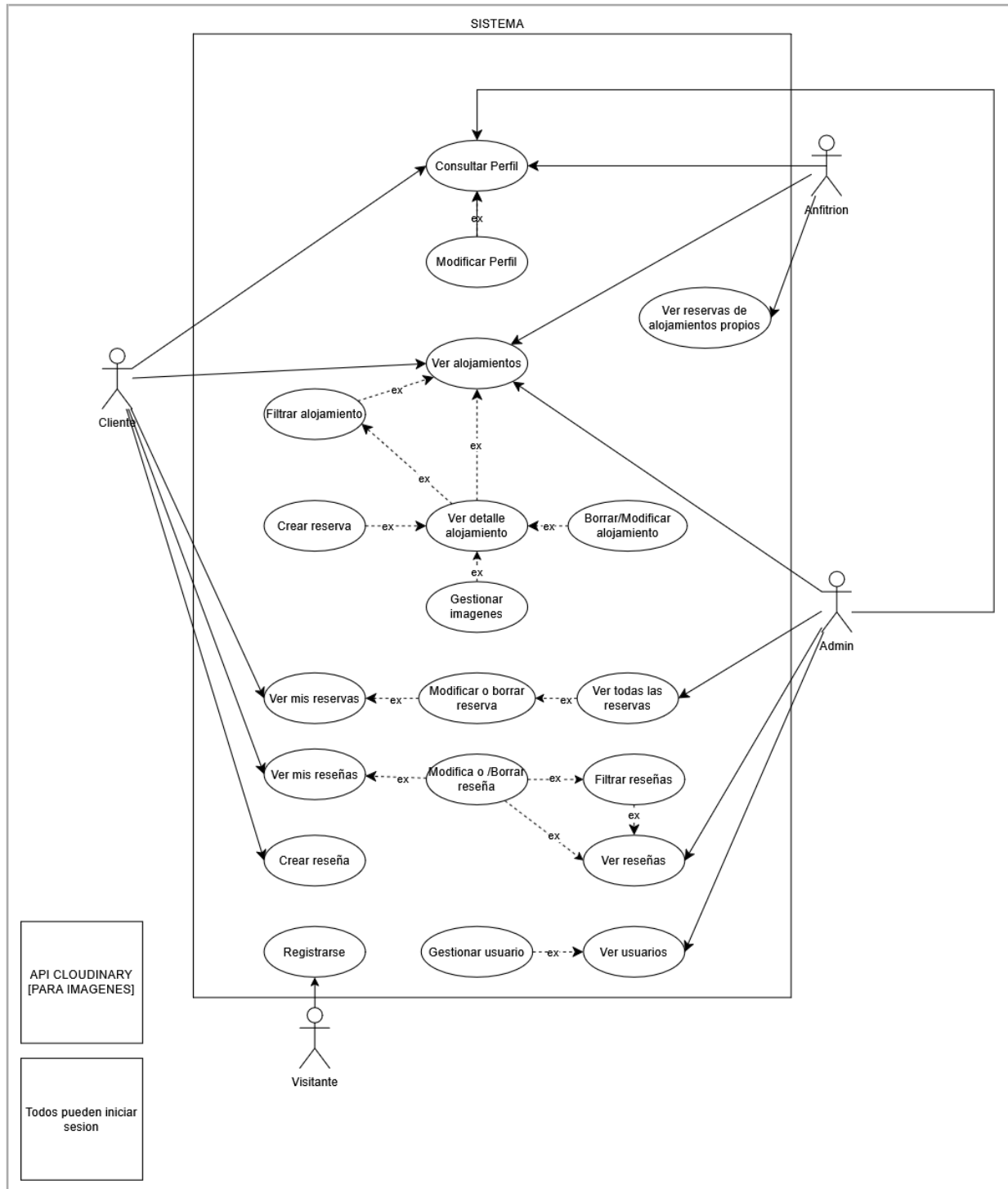
- **RF21:** El sistema debe permitir al administrador gestionar la información almacenada en el sistema.

3.2 Requisitos no funcionales

- **RNF01:** El sistema debe contar con una interfaz web intuitiva y fácil de utilizar.
- **RNF02:** El sistema debe garantizar la autenticación de usuarios mediante credenciales seguras (JWT).
- **RNF03:** El sistema debe restringir el acceso a funcionalidades según el rol del usuario (CLIENTE, ANFITRION, ADMINISTRADOR).
- **RNF04:** El sistema debe almacenar la información de forma persistente en una base de datos relacional (MySQL).
- **RNF05:** El sistema debe mantener la integridad de los datos durante las operaciones de alta, baja y modificación.
- **RNF06:** El sistema debe responder a las solicitudes del usuario en tiempos adecuados para una experiencia fluida.
- **RNF07:** El sistema debe ser accesible desde navegadores web modernos.
- **RNF08:** El sistema debe permitir la escalabilidad y el mantenimiento mediante una arquitectura en capas (Controller – Service – Repository).
- **RNF09:** El sistema debe documentar los servicios REST mediante Swagger / OpenAPI.
- **RNF10:** El sistema debe utilizar mecanismos de validación para garantizar la consistencia de los datos ingresados.
- **RNF11:** El sistema debe proteger la información sensible de los usuarios durante la autenticación.
- **RNF12:** El sistema debe permitir la reutilización y mantenimiento del código mediante el uso de buenas prácticas de desarrollo (DTOs, servicios, repositorios).

4. Diagrama de casos de uso

A continuación se presenta el diagrama de casos de uso del sistema AlojaME, identificando los actores principales y sus interacciones con el sistema.



Los actores identificados en el sistema son:

- Visitante: usuario no registrado que puede registrarse en el sistema.
- Cliente: usuario registrado que puede buscar alojamientos, realizar y gestionar reservas, y escribir reseñas.
- Anfitrión: usuario registrado que puede gestionar sus alojamientos, galerías de imágenes y consultar reservas de sus propiedades.
- Administrador: usuario con privilegios totales para gestionar usuarios y contenido del sistema.

4.1 Especificación de caso de uso

Caso de Uso: Realizar reserva de alojamiento

Actor principal: Cliente

Precondiciones: El usuario debe estar autenticado con rol CLIENTE y el alojamiento debe estar disponible para las fechas seleccionadas.

Postcondiciones: La reserva queda registrada en el sistema con estado PENDIENTE.

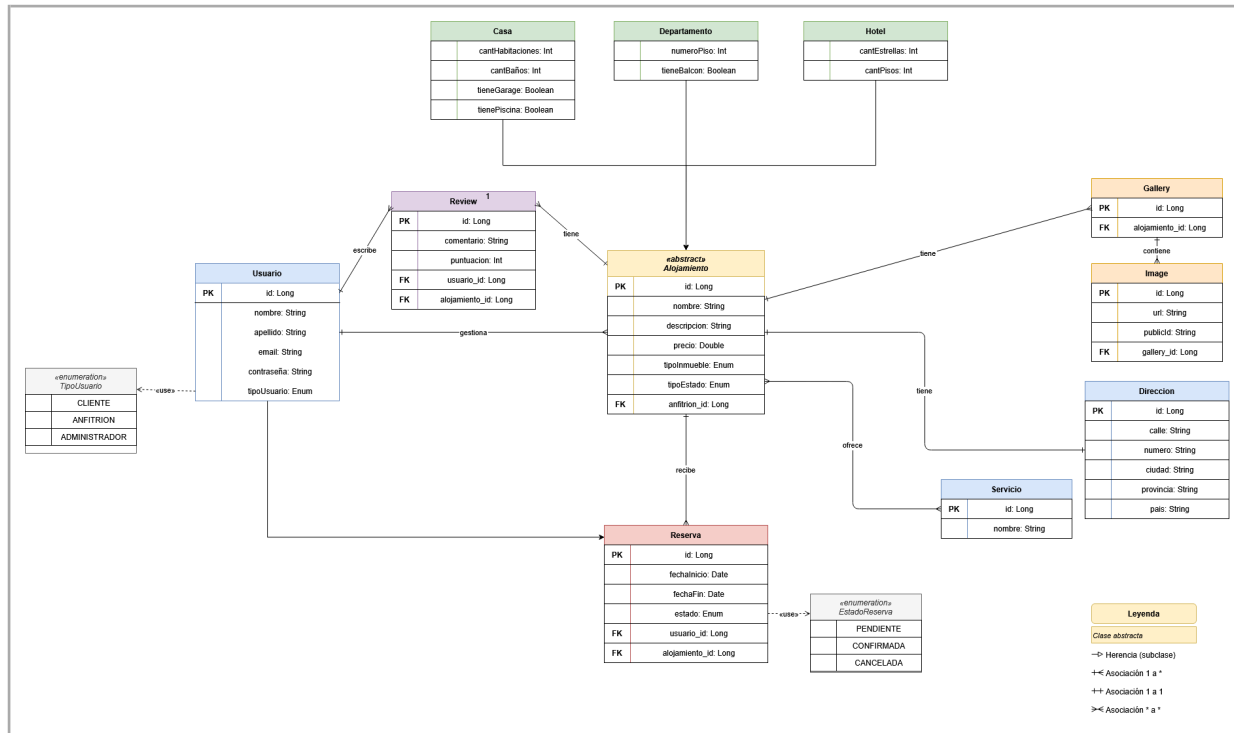
Flujo principal:

1. El cliente navega al listado de alojamientos disponibles.
2. El cliente selecciona un alojamiento y visualiza su detalle.
3. El cliente ingresa las fechas de inicio y fin de la reserva.
4. El sistema valida la disponibilidad del alojamiento para las fechas indicadas.
5. El sistema registra la reserva y la asocia al usuario autenticado.
6. El sistema muestra confirmación de la reserva al cliente.

Flujo alternativo: Si las fechas están ocupadas, el sistema informa al cliente y solicita nuevas fechas.

5. Modelo conceptual

El modelo conceptual representa las entidades principales del dominio del sistema y sus relaciones.



Entidades principales del sistema:

- Usuario: representa a todos los actores del sistema. Atributos: id, nombre, apellido, email, contraseña, tipoUsuario (CLIENTE, ANFITRION, ADMINISTRADOR).
- Alojamiento (clase abstracta): base de Casa, Departamento y Hotel. Atributos comunes: id, nombre, descripción, precio, dirección, tipoInmueble, tipoEstado, anfitrión.
- Casa: hereda de Alojamiento. Atributos específicos: cantidadHabitaciones, cantidadBaños, tieneGarage, tienePiscina.
- Departamento: hereda de Alojamiento. Atributos específicos: numeroPiso, tieneBalcon.
- Hotel: hereda de Alojamiento. Atributos específicos: cantidadEstrellas, cantidadPisos.
- Reserva: relaciona un Usuario (cliente) con un Alojamiento. Atributos: id, fechaInicio, fechaFin, estado, usuario, alojamiento.
- Review: reseña de un usuario sobre un alojamiento. Atributos: id, comentario, puntuación, usuario, alojamiento.
- Direccion: dirección física del alojamiento. Atributos: calle, número, ciudad, provincia, país.
- Gallery / Image: entidades para gestión de galería de imágenes mediante Cloudinary.
- Servicio: servicios disponibles asociados a alojamientos.

Relaciones entre entidades:

- Usuario (1) — tiene → (*) Alojamiento (como anfitrión)
- Usuario (1) — realiza → (*) Reserva
- Alojamiento (1) — recibe → (*) Reserva
- Usuario (1) — escribe → (*) Review
- Alojamiento (1) — tiene → (*) Review
- Alojamiento (1) — tiene → (1) Direccion
- Alojamiento (1) — tiene → (*) Gallery
- Gallery (1) — contiene → (*) Image